

ETMS Employee Ticket Management System

Project Documentation

Field	Value
Version	1.0.0
Last updated	2026-05-04
Source of truth	repository files only (no external assumptions)

Table of Contents

- 1. Executive Summary
- 2. Project Architecture
- 3. Technology Stack
- 4. Repository and Folder Structure
- 5. Getting Started
- 6. Environment Configuration
- 7. Database Documentation
- 8. Backend API Documentation
- 9. User Roles and Permissions
- 10. Business Logic and Workflows
- 11. Frontend Documentation
- 12. Core Services
- 13. Middleware and Utilities
- 14. Data Models and Type Definitions
- 15. Security
- 16. File Management
- 17. Background Jobs
- 18. Testing
- 19. Deployment
- 20. Configuration Reference
- 21. Troubleshooting
- 22. API Integration Guide
- 23. Data Import and Export
- 24. Maintenance and Operations
- 25. Development Guidelines
- 26. Recent Updates and Fixes
- 27. Future Enhancements
- 28. Appendices

1. Executive Summary

ETMS (Employee Ticket Management System) is a full-stack platform for creating, approving, assigning, and resolving internal IT and operational requests. The system enforces role-based workflows, tracks asset ownership, and monitors SLA deadlines. It also provides a Data Portal flow for data-specific requests and responses.

What the system delivers:

- Ticket creation with category, priority, SLA days, optional attachments, and asset linking.
- Multi-step approval and escalation for request tickets.
- Auto-assignment (round-robin) to employees within a mapped team.
- Asset lifecycle tracking with transfer history and spec details.
- Data Portal requests with response file download.
- SLA escalation job with email notification.

Current state based on repository notes: backend service modules are centralized, access control is formalized, and SLA escalation tests are verified.

2. Project Architecture

2.1 High-Level Architecture

- Frontend (React + Vite) calls a REST API over JSON and multipart/form-data.
- Backend (Express + TypeScript) performs validation, access control, and persistence.
- PostgreSQL stores normalized ticket, asset, user, and audit data.
- Uploaded files are stored on disk and referenced in the database.
- Email is sent via configured SMTP or Ethereal in local dev.
- A cron job evaluates SLA due dates and triggers escalation alerts.

2.2 Major Components

- Frontend SPA with role-based routing and UI permissions.
 - REST API with auth, ticketing, approvals, assets, user management, reports, and data portal.
 - Database schema defined in raw SQL with enums, constraints, and indexes.
 - Background job for SLA escalation.
 - Swagger UI served by the backend for OpenAPI documentation.
-

3. Technology Stack

Frontend

- React 18
- Vite 5
- TypeScript 5
- Tailwind CSS 3
- Axios
- React Router DOM
- date-fns

Backend

- Node.js
- Express
- TypeScript
- PostgreSQL (pg)
- JWT (jsonwebtoken)
- bcrypt
- multer (file upload)
- nodemailer
- node-cron
- helmet, cors, morgan
- swagger-ui-express + yaml (OpenAPI rendering)

Database

- PostgreSQL
 - Raw SQL schema, migrations, and seed data (database/*.sql)
-

4. Repository and Folder Structure

Top-level documentation files:

- README.md: overall project overview and structure
- README_BACKEND_FIXES.md: backend improvements
- DEPLOYMENT_CHECKLIST.md: deployment steps
- MANAGER_SERVICE_IMPLEMENTATION_COMPLETE.md: manager service notes
- SLA_ESCALATION_TEST_RESULTS.md: SLA test outcomes

Core workspace layout:

- etms-mvp/
 - client/: frontend app
 - server/: backend API
 - database/: schema + seeds + migrations
 - scripts/: helper scripts
-

5. Getting Started

5.1 Prerequisites

- Node.js 18+
- PostgreSQL 14+
- npm

5.2 Database Setup

The SQL files are authoritative. Use the schema and seed scripts from etms-mvp/database:

```
psql -U postgres -c "CREATE DATABASE uiicdb;"
psql -U postgres -d uiicdb -f etms-mvp/database/schema.sql
psql -U postgres -d uiicdb -f etms-mvp/database/seed.sql
```

Note: Root scripts in etms-mvp/package.json reference a database named ETMS. Align DB_NAME and scripts to the same database name to avoid confusion.

5.3 Backend

```
cd etms-mvp/server
npm install
npm run dev
```

Additional backend scripts:

```
npm run build
npm run typecheck
```

Current local endpoints:

- API base: <http://localhost:5003/api>
- Swagger UI: <http://localhost:5003/api/docs>

5.4 Frontend

```
cd etms-mvp/client
npm install
npm run dev
```

Additional frontend scripts:

```
npm run build
npm run typecheck
```

Vite defaults to <http://localhost:5173>. If 5173 is in use, Vite selects the next available port (for example 5174).

5.5 Run Client and Server Together

From etms-mvp/:

```
npm run dev
```

Scripts (etms-mvp/package.json):

- dev:server
- dev:client
- dev
- setup:db
- seed:db
- test:manager
- test:data-portal-routing
- test:ticket-cycle-consistency
- test:reported-transition

6. Environment Configuration

6.1 Server Environment Variables

Required (validated at startup):

- DB_HOST
- DB_PORT
- DB_NAME
- DB_USER
- DB_PASSWORD
- JWT_SECRET (min 32 chars recommended)

Optional (current server/.env values included in parentheses):

- PORT (5003)
- CLIENT_URL (<http://localhost:5173>)
- UPLOAD_DIR (./uploads)
- RESPONSE_UPLOAD_DIR (./uploads/responses)
- SMTP_HOST (blank uses Ethereum)
- SMTP_PORT (587)
- SMTP_USER
- SMTP_PASS
- MAIL_FROM (ETMS noreply@uiic.co.in)
- SLA_CRON_SCHEDULE (0 0 * * *)

6.2 Client Environment Variables

Client .env.example:

```
VITE_API_URL=http://localhost:5000/api
```

If backend is on 5003, update the runtime client config to:

```
VITE_API_URL=http://localhost:5003/api
```

7. Database Documentation

7.1 ENUM Types

- user_role_enum: employee, manager, admin, data_team
- priority_enum: low, medium, high, critical

7.2 Core Tables (schema.sql)

1. users

- id, emp_id, name, email, password_hash
- role (user_role_enum)
- category_id (FK -> ticket_categories)
- team, is_active, password_changed_at
- created_at

2. ticket_types

- id, name, type_key (complaint, request, data)

3. ticket_categories

- ticket_type_id (FK -> ticket_types)
- name, category_key
- default_priority (priority_enum)
- manager_user_id (FK -> users)
- assigned_team_key, is_team, requires_approval

4. assets

- name, serial_number, category_id (FK -> ticket_categories)
- assigned_to (FK -> users), status (active|under_repair|retired)
- spec columns: machine_type, model, ram, hdd, monitor_serial, monitor_make, system_ip, port, ms_office_ver, os, host_id, floor, branch
- created_at, updated_at

5. asset_assignments

- asset_id (FK -> assets), from_user_id, to_user_id
- transferred_by, assigned_at, returned_at, note
- check: returned_at is null or >= assigned_at

6. tickets

- ticket_no format UIIC-{C|R|D}-YYYY-XXXXXX
- title, description, ticket_type_id, category_id, priority
- status (pending_approval, approved, assigned, in_progress, reported, resolved, closed, rejected)
- raised_by, assigned_to, approval_owner_id, asset_id
- rejection_reason, report_reason
- sla_days (1-30), sla_due_date
- escalated, escalated_at
- created_at, updated_at

7. attachments

- ticket_id (FK -> tickets)
- filename, original_name, file_path, file_size, mime_type
- uploaded_by, uploaded_at

8. ticket_logs

- ticket_id, action, old_status, new_status, performed_by, note
- created_at

7.3 Indexes and Constraints (highlights)

- tickets: raised_by, assigned_to, approval_owner_id, status, type, category, asset, SLA due date
- assets: serial_number, category_id, assigned_to, status
- asset_assignments: unique active assignment per asset
- ticket_categories: team_key, is_team, manager_user_id

7.4 Additional SQL Scripts

- create_permissions_system.sql: permissions tables and helper functions
- create_manager_assignment_history.sql: manager_assignment_history table
- patch_data_team_routing.sql: data portal routing updates
- migrate_asset_spec_columns.sql: asset spec columns
- fix_categories.sql: category normalization
- add_category_employees.sql: category employee inserts
- seed_two_managers.sql: two-manager seed variant

8. Backend API Documentation

Base URL (current): <http://localhost:5003/api> Swagger UI: <http://localhost:5003/api/docs> OpenAPI source: [docs/openapi.yaml](#)

Auth: JWT via Authorization: Bearer

8.1 Authentication

- POST /api/auth/login (Login)
- GET /api/auth/me (Current user)

8.2 Tickets

- POST /api/tickets (Create ticket, multipart/form-data)
- GET /api/tickets (List tickets with filters)
- GET /api/tickets/:id (Ticket details)
- GET /api/tickets/:id/allowed-statuses (Status transitions)
- PUT /api/tickets/:id/status (Update status)
- GET /api/tickets/:id/file (Download attachment)
- POST /api/tickets/:id/response-file (Upload response file)
- GET /api/tickets/:id/response-file (Download response file)

8.3 Approvals

- GET /api/approvals/pending
- POST /api/approvals/:ticketId/approve
- POST /api/approvals/:ticketId/reject
- POST /api/approvals/:ticketId/reapprove

8.4 Assets

- GET /api/assets
- GET /api/assets/my
- GET /api/assets/:id
- GET /api/assets/:id/history
- POST /api/assets
- PATCH /api/assets/:id/spec
- PATCH /api/assets/:id/status
- POST /api/assets/:id/transfer
- DELETE /api/assets/:id

8.5 Users

- GET /api/users
- POST /api/users
- PATCH /api/users/change-password
- PATCH /api/users/:id/toggle-active
- PATCH /api/users/:id/name
- POST /api/users/transfer-ownership
- DELETE /api/users/:id

8.6 Categories and Teams

- GET /api/categories
- GET /api/categories/:categoryId/employees
- GET /api/categories/teams
- POST /api/categories/teams
- DELETE /api/categories/teams/:teamId

8.7 Reports

- GET /api/reports/top-failing-devices (limit 1-50)

8.8 Lookup

- GET /api/lookup/employee?q=EMP_ID_OR_SERIAL

8.9 Import

- POST /api/import/assets/excel (admin/manager, .xls/.xlsx)

8.10 Data Portal

- GET /api/data-portal/teams
- POST /api/data-portal/tickets
- GET /api/data-portal/tickets
- GET /api/data-portal/tickets/:id
- GET /api/data-portal/tickets/:id/response-file

8.11 System

- GET /health (health check)
-

9. User Roles and Permissions

Roles:

- employee
- manager
- admin
- data_team

Access control rules observed in code and documentation:

- Admin: full access but cannot change ticket status.
- Manager: approvals and re-approvals within managed categories.
- Employee: can act on assigned tickets and raise new tickets.
- Data Team: restricted to Data Portal flows.

Asset-specific rules:

- Asset management is constrained to infra managers; standard employees have read-only access to their assigned assets.
-

10. Business Logic and Workflows

10.1 Ticket Lifecycle and Transitions

Lifecycle order:

pending_approval -> approved -> assigned -> in_progress -> resolved -> closed

Escalation paths and terminal states:

- reported sends a request ticket back to pending_approval
- rejected and closed are terminal

Transition enforcement is defined in ticketTransitions.ts and includes role-specific checks.

10.2 Approval Workflow

- Managers approve or reject pending tickets.
- Approved tickets move to assigned via system auto-assignment.
- Escalated tickets require explicit re-approval.

10.3 Assignment Logic

- Assignment is round-robin based on open ticket count in the mapped team.

10.4 Data Ticket Flow

- Data tickets require a response file before resolve actions are valid.

10.5 SLA Escalation

- assigned or in_progress tickets past sla_due_date are escalated.
- Escalations are emailed to category managers.
- A 24-hour re-escalation rule limits repeat notifications.

10.6 Manager Assignment Tracking

- ManagerService centralizes manager lookups and caching.
 - Assignment history is recorded for audit in manager_assignment_history.
-

11. Frontend Documentation

11.1 Routing Map

- /login
- /change-password
- /dashboard
- /tickets
- /tickets/new
- /tickets/:id
- /approvals
- /reports
- /assets
- /admin
- /admin/teams
- /data-portal

11.2 Key Pages

- LoginPage: login and first-time password change detection.

- ChangePasswordPage: forced password update.
- DashboardPage: ticket KPIs by role.
- TicketsPage: filterable ticket list.
- NewTicketPage: ticket creation flow with category and asset steps.
- TicketDetailPage: ticket details, logs, transitions, uploads.
- PendingApprovalsPage: manager approval queue.
- ReportsPage: top failing devices report.
- AssetsPage: asset list (read-only for employees, full management for infra managers).
- AdminPage: user, asset, and ticket administration tabs.
- AdminTeamsPage: team create/delete and member management.
- DataPortalPage: data ticket creation and downloads.

11.3 Frontend Structure Highlights

- api/axiosInstance.ts: Axios client with JWT interceptor and 401 auto-logout.
- api/authApi.ts: login, getMe, changePassword calls.
- api/ticketApi.ts: tickets, assets, users, approvals, reports, import, lookup.
- context/AuthContext.tsx: JWT state stored per tab in sessionStorage.
- components/common: StatusBadge, PriorityBadge, TypeBadge.
- components/tickets: TicketCard, TicketTypeSelector, CategorySelector.
- components/admin: CategoryOrgGraph for manager-employee visualization.

12. Core Services

- ManagerService: manager lookup, assignment history, workload metrics, caching.
- managerAssignmentService: legacy wrapper around ManagerService.
- accessControlService: permission checks and RBAC logic.
- assignmentService: round-robin assignee selection.
- emailService: templated notification logic.

13. Middleware and Utilities

Middleware

- authMiddleware: JWT verification and user loading.
- roleMiddleware: requireRole(...) checks.
- uploadMiddleware: multer disk storage with MIME type filtering and size cap.

Utilities

- jwtUtils: JWT sign/verify with 8-hour expiry.
- generateTicketId: ticket number generation.
- ticketTransitions: central transition validation.
- sanitizeFilename: safe upload filenames.
- fileCleanup: safe deletion utilities.
- validateEnv: environment validation at startup.

14. Data Models and Type Definitions

Backend types (server/types/index.ts):

- UserRow, TicketRow, CategoryRow, AssetRow, TicketLogRow, AttachmentRow

Frontend types (client/src/types/index.ts):

- User, Ticket, Category, Asset, TicketLog, Attachment

These types mirror database row structures and API payloads.

15. Security

- JWT-based auth (8h expiry).
 - bcrypt hashing (10 rounds).
 - Environment validation on startup.
 - helmet + cors for HTTP security and origins.
 - Role-based access enforcement.
 - File uploads restricted by MIME type and 5 MB size limit for ticket attachments.
-

16. File Management

- Request attachments saved to uploads/.
 - Data response files saved to uploads/responses/.
 - Files are stored with sanitized names and referenced in attachments.
 - Downloads served through specific ticket endpoints.
-

17. Background Jobs

SLA Escalation Job:

- Schedule controlled by SLA_CRON_SCHEDULE.
 - Default schedule: 0 0 * * * (midnight daily).
 - Escalates overdue assigned/in_progress tickets and emails managers.
-

18. Testing

Server scripts:

- server/scripts/testManagerService.ts
- server/scripts/testAccessControl.ts
- server/scripts/testSlaEscalation.ts
- server/scripts/testSlaEscalationViaServer.ts
- server/scripts/verifyManagerService.ts

Root scripts (etms-mvp/package.json):

- test:manager
 - test:data-portal-routing
 - test:ticket-cycle-consistency
 - test:reported-transition
-

19. Deployment

Reference DEPLOYMENT_CHECKLIST.md. Key steps:

- Build backend and frontend.
 - Configure .env values securely.
 - Ensure upload directories exist.
 - Apply schema and migrations.
 - Validate /health endpoint.
-

20. Configuration Reference

Backend:

- PORT
- CLIENT_URL
- DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
- JWT_SECRET
- SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS
- MAIL_FROM
- UPLOAD_DIR, RESPONSE_UPLOAD_DIR
- SLA_CRON_SCHEDULE

Frontend:

- VITE_API_URL
-

21. Troubleshooting

Common issues and checks:

- Port in use: change PORT in server/.env or stop the conflicting process.
 - 401 responses: verify JWT token and JWT_SECRET.
 - File upload errors: ensure MIME types match allowed list and size limit.
 - Missing env vars: server exits early due to validateEnv.
 - Data ticket resolution blocked: upload response file before resolve.
 - Client cannot reach API: align VITE_API_URL with backend port.
-

22. API Integration Guide

1. POST /api/auth/login to get a JWT token.
2. Store token in sessionStorage and attach it as Authorization: Bearer .

3. For file endpoints, use multipart/form-data with a file field named file.
 4. Handle 401 by clearing session and redirecting to /login.
-

23. Data Import and Export

- Asset import: POST /api/import/assets/excel (.xls/.xlsx)
 - Data portal downloads: GET /api/data-portal/tickets/:id/response-file
 - Ticket attachments: GET /api/tickets/:id/file
 - Data response files: GET /api/tickets/:id/response-file
-

24. Maintenance and Operations

- Monitor server logs for SLA jobs and email delivery.
 - Backup database and uploads directories regularly.
 - Review ticket_logs and asset_assignments for audit history.
-

25. Development Guidelines

- Keep role strings in constants/ROLES.ts.
 - Use ticketTransitions.ts for status rules.
 - Prefer accessControlService for permission checks.
 - Keep API contract in sync with docs/openapi.yaml.
-

26. Recent Updates and Fixes

Based on repository notes:

- Environment validation added.
 - JWT expiration set to 8 hours.
 - Upload directory creation and cleanup improvements.
 - Manager Service centralized with caching and history.
 - Access control service and RBAC introduced.
 - /health endpoint added.
 - SLA escalation system tested and verified.
 - Swagger/OpenAPI documentation wired to /api/docs.
-

27. Future Enhancements

Recommended by existing repo notes:

- Rate limiting
- Structured logging
- Improved workload balancing
- Expanded automated tests
- Team member management

- UI: admin team screen to add, remove, and transfer members across teams with confirmation dialogs and audit notes.
 - API: endpoints for add, remove, and transfer team members with validation, permissions, and audit logging.
-

28. Appendices

28.1 Ticket Status Reference

- pending_approval
- approved
- assigned
- in_progress
- reported
- resolved
- closed
- rejected

28.2 Default Login Accounts (seed.sql)

- admin@uiic.co.in
- mgr.infra@uiic.co.in
- mgr.network@uiic.co.in
- data.portal@uiic.co.in

28.3 File Upload Constraints

- Allowed MIME types: pdf, doc, docx, txt, png, jpg
- Max attachment size: 5 MB

End of document.